



AI 短剧工厂开箱指南

一条推文 8 个开源项目全验货 · 深挖 28.5k★ Pixelle-Video · 零 API 费离线跑通"主题→成片"流水线

清单 8/8 全真

Pixelle-Video 28,581★

实验真实产出 28.36s 成片

2026-10-03

这份指南的来历

推主 @denziideng 发帖整理了 8 个 GitHub 高星 AI 短剧项目。我们用 GitHub API 逐个验货：8/8 全部真实存在，其中 2 个链接因组织改名已 301 重定向（[saturndec→waooAI](#)、[AIDC-AI→ATH-MaaS](#)），并非假仓库。深挖其中最高星的 **Pixelle-Video**（阿里国际 AIDC 团队出品，Apache-2.0），其余 7 个做成选型对比。

Q1 一条"主题"是怎么变成一部片的？

把视频创作拆成**工厂流水线**：过去你要自己写文案、找图师、请配音、开剪辑软件；Pixelle-Video 把这四件事变成**六个自动衔接的工序**，你只在第一站投喂一个词，比如"为什么人工智能短剧正在爆火"。

你只投喂一个主题 ▼ 出来一部成片



对应源码：pixelle_video/pipelines/standard.py (StandardPipeline)

流水线 (Pipeline)：把大任务切成顺序工序，上一站的产出就是下一站的原料——和你知识库 [[构建流水线与Pass]] 是同一思想，只是这次的"工件"从代码变成了视频帧。

Q2 为什么整条流水线围着一张"分镜表"转？

流水线各站之间传的不是视频文件，而是一份 **Storyboard (分镜表) JSON**——这是全项目最重要的数据结构。每个 **StoryboardFrame** 记录一帧的"剧本、画面描述、素材路径、时长"，六个工序都在往这张表里填空。

StoryboardFrame × N

narration: "十年前，做一支视频需要..."
image_prompt: "cinematic wide shot, ..."
audio_path: audio_0.mp3 ← TTS 站填
image_path: frame_0.png ← 配图站填
duration: 4.94s ← 合成站回填

文案站：只写前两个字段

素材站：补齐 path 字段

合成站：回填时长，表满=片成

这就是知识库里的 [[提示词模板]] 思路再进一步：**LLM 的输出被结构化 JSON 钉死**（每句解说 5~20 词、每条图像提示词 30~60 词，超了重来），后面所有工序只认字段不认人话。结构化中间表 = 流水线的**传送带**。

Q3 8 个项目共同的死穴：角色"变脸"问题

AI 生视频最致命的不是画质也不是配音，而是**同一角色换个镜头就换张脸**——模型每次生成都是独立抽签。8 个项目里有一半的主打功能都在打这一个点。

① 定妆照锚点

给角色生成设定图
每次生成都参考它

moyin-creator
6层身份锚点

② 首尾关键帧

镜头起点终点先画死
模型只补中间插值

BigBanana
Keyframe工作流

③ 资产复用制

角色/场景/服装入库
跨镜头跨集只引用

Jellyfish
一致性一等公民

④ 参考图注入

真实照片/宠物照
钉进每一帧生成

ViMax
AutoCameo

四派思路殊途同归：先固定"是谁/在哪"，再让模型自由发挥剩下的部分

Q4 "抽卡式"与"工业化"两代工具的代差在哪？

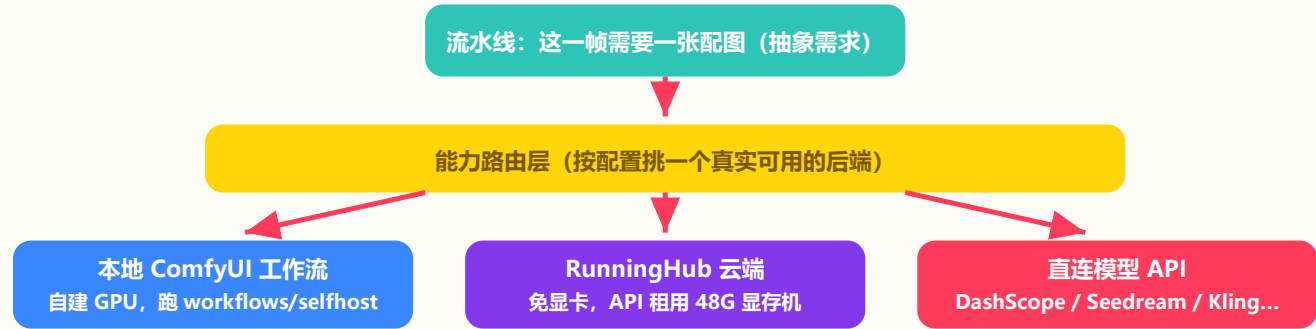
BigBanana README 里那句暴论值得记住：**"先有演员和片场，再开机"**。8 个项目实际分成两代：

维度	第一代：抽卡式（一键生成）	第二代：工业化（Script-to-Asset-to-Keyframe）
代表	Pixelle-Video / ViMax / huobao-drama（解说向）	BigBanana / Jellyfish / moyin-creator / Toonflow（剧情向）
工作方式	输入主题→LLM 直接扩写→每帧独立生成	剧本→角色/场景资产入库→关键帧锁定→镜头生成
角色稳定性	弱：每帧是独立抽签	强：定妆照+衣橱+上下文三重锁定
适合内容	解说/科普/书单号短视频（旁白为主，人物少）	连载短剧/漫剧/网文改编（角色要跨集存活）
上手成本	低：一句话出片	高：要做分镜和台词微调

一句话记忆：**解说视频拼的是产能，剧情短剧拼的是一致性**。选工具先问自己"我的角色需要跨镜头存活吗"。

Q5 图像/视频后端凭什么说换就换？

Pixelle-Video 的招牌是 **原子能力灵活组合**：文案、配图、视频、语音每个环节都是独立"原子"，各配一张后端菜单。你的图像可以走本地 ComfyUI 工作流、走 RunningHub 云端 48G 显存机器、或直连 DashScope/Seedream API——**改配置不改代码**。



这正是知识库里的 [[能力层与后端路由]]：流水线只提"抽象需求"，路由层按配置顺序挑第一个真实可用的后端。模型换代（WAN 2.1 → Seedance 2.0）只动配置表，流水线代码零改动。上游甚至把同一能力封装成 **MCP 工具**（依赖 fastmcp），别的 Agent 也能调它出片。

Q6 免费配音是哪来的？edge-tts 的秘密

Pixelle-Video 默认 TTS 引擎是 [edge-tts](#) (pyproject 里锁死 7.2.7 版本)。它调用的是 **Microsoft Edge 浏览器"大声朗读"功能的 WebSocket 接口**——就是 Edge 里右键网页"朗读此内容"用的那套神经语音，被逆向封装成了 Python 库，所以免费、免 key、音色还是商业级的。

```
python -c "import asyncio, edge_tts; \
asyncio.run(edge_tts.Communicate('你好短剧', 'zh-CN-XiaoxiaoNeural').save('hi.mp3'))"
# 3 秒后得到 hi.mp3 —— 一行代码，商业级中文女声，0 元
```

上游支持两条 TTS 路线：**local** (edge-tts 免费直连) 和 **comfyui** (工作流跑 Index-TTS 等，支持参考音频克隆音色)。免费路线够用时，别急着上付费的。

Q7 一张 PNG + 一段 MP3，怎么变成 MP4？

ffmpeg 三板斧，正是上游 `services/video.py` 干的活：

板斧	命令核心	干什么
① loop 定格	<code>-loop 1 -i frame.png -i audio.mp3 -shortest</code>	静态图循环播放，配上一句语音， -shortest 让段长=语音长 ，一帧变一段
② concat 拼接	<code>-f concat -safe 0 -i list.txt -c copy</code>	concat demuxer 按清单 无损拼接 (-c copy 不重编码，秒级完成)
③ amix 混音	<code>-filter_complex amix ... volume=0.15</code>	旁白 + BGM (音量压到 15%) 混成一条音轨

⚠ **实验亲踩的坑**：混音时如果只把 `amix` 的音频流传给输出，**视频流会被整个丢掉，成片变成"音频文件"**。必须 `ffmpeg.output(video_stream, mixed_audio, ...)` 两路都写。第二个坑：`ffmpeg-python` 的两个 `input()` 不能链式连写，要先分别建流再一起传给 `output()`。

Q8 代码架构长什么样？为什么好读？

仓库仅约 2 万行 Python，五层结构清晰：**api/** (FastAPI 路由) + **web/** (Streamlit 双前端) → **pipelines/** (流水线编排) → **services/** (TTS/LLM/视频等原子能力) → **models/** (Storyboard 数据类)。

编排层用的是模板方法模式：LinearVideoPipeline 把流水线钉成 8 个生命周期钩子 (setup→content→title→visuals→storyboard→assets→post→finalize)，子类 StandardPipeline / AssetBasedPipeline / CustomPipeline 各自只覆写需要的钩子。想加一条新流水线 = 继承后改 2~3 个钩子，这就是它 5 条流水线却不显乱的原因。



Q9 8 个项目验货报告与选型指南

项目	★	定位一句话	技术栈	注意点
Pixelle-Video ★深挖	28,581	主题一句话→解说短视频，流水线最干净	Python/FastAPI/Streamlit/ffmpeg	Apache-2.0; 配图/视频环节仍需模型后端
Toonflow	16,339	无限画布+Agent+可视化 workflow，21 国语言	TypeScript/Bun	偏平台级，体量大
huobao-drama	15,630	一句话生成完整短剧（剧本→成片）	Nuxt3+Hono+Mastra+SKILL.md	CC BY-NC-SA 禁商用; 强绑 firemux API key
waoowao	14,346	画布式 AI 影视工作台（Docker 自托管）	Docker/Caddy/NextAuth	preview 阶段; 安装要自签 TLS
ViMax	12,531	学术派 Agentic 视频: Idea/Script/Novel→Video	Python/Agent Loop/uv	HKUDS 出品, 含论文与技术报告
Jellyfish	6,595	短剧制片工作台，资产一致性一等公民	异步任务系统+OpenAPI	2026-07 后停更
moyin-creator	4,639	生产级批量化，主打 Seedance 2.0	多模态引用+参数校验	深度依赖付费视频模型
BigBanana	2,257	反抽卡工业化漫剧工作台	浏览器 IndexedDB 本地存储	配套付费课程营销较多

验货结论：8/8 真实，无假货。2 个链接过期但 301 跳转可达（GitHub 组织改名：saturndec→waoowaoAI、AIDC-AI→ATH-MaaS——原阿里国际 AIDC 团队项目群整体迁移）。新手首选 Pixelle-Video（架构最干净、文档全、免费 TTS 可玩），连载剧情创作选 BigBanana/Jellyfish 路线。

Q10 学完我能干什么？

1. **零成本做解说短视频**：edge-tts 配音 + 本地合成，配图用你已有的 API key 即可出片（书单号/科普号直接可用）；
2. **把流水线当模板抄**：Storyboard 传送带 + 模板方法钩子，可平移到任何"批量生成"场景（批量海报、批量播客）；
3. **给别的 Agent 当手脚**：项目自带 fastmcp，可把"出片"注册成 MCP 工具，让 Claude/其他 Agent 一句话调它；
4. **选型不踩坑**：8 个项目的两代分野与许可证红线（如 huobao 禁商用）已在手；
5. **验货方法论**：GitHub API 查 301 重定向——链接失效 ≠ 项目造假，转发清单前先 API 验一遍。



动手实验：零 API 费离线复刻流水线

目标：不装 ComfyUI、不买任何 API key，用 5 个脚本走完"主题→成片"全流程，产出一条 **28.36 秒、846 帧、带 BGM 的竖屏成片**。LLM 写文案这一环用预写剧本代替（离线），其余每一环都真实执行。环境：Python 3.11+，`pip install edge-tts==7.2.7 ffmpeg-python pillow imageio-ffmpeg`（imageio-ffmpeg 自带静态 ffmpeg，无需系统安装）。

Step 1 | ex1_storyboard.py —— 生成"传送带"分镜表

操作：`python ex1_storyboard.py` **目的**：拿到流水线的结构化中间表示（字段 1:1 对齐上游 StoryboardFrame）

实现：1 个主题 + 5 帧解说词/图像提示词 → `output/storyboard.json`

✅ **预期结果**：打印 5 帧文案并以 `[ex1] PASS` 收尾；storyboard.json 里每帧有 narration 与 image_prompt 字段。

Step 2 | ex2_frames.py —— PIL 离线"配图 + 烧字幕"

操作：`python ex2_frames.py` **目的**：代替 ComfyUI 出图环节，体验"帧模板合成"

实现：5 张 1080×1920 渐变底图，主题大字 + 底部字幕直接画进图（对应上游 composed_image_path 的烧字幕逻辑）

✅ **预期结果**：`[ex2] PASS: 5 张 1080x1920 配图生成`；改 ACCENTS 里任意色值重跑，图立刻变装（参数→产物可见变化）。

Step 3 | ex3_tts.py —— edge-tts 真实联网配音

操作: `python ex3_tts.py` 目的: 与上游同库同版本 (7.2.7) 合成每帧旁白

实现: zh-CN-XiaoxiaoNeural 音色、+8% 语速, 5 段 mp3 落盘并回填 audio_path

✅ 预期结果: 5 个 audio_*.mp3, 29~40KB; `[ex3] PASS: 5 段真实微软 TTS 语音合成`。实测输出: audio_0.mp3 29,664 bytes ... audio_4.mp3 30,096 bytes。

Step 4 | ex4_compose.py —— ffmpeg 三板斧出片

操作: `python ex4_compose.py` 目的: 复刻上游逐帧合成→拼接→BGM 后期

实现: ① loop+shortest 切出 5 段 (4.94/6.02/6.72/5.59/5.02s) ② concat -c copy 无缝拼 ③ 上游自带 `bgm/default.mp3` 以 0.15 音量 amix 混入

✅ 预期结果: 预期总时长≈28.29s, 成片实际 28.36s, `final_video.mp4 (590 KB)`。踩坑记录: 混音漏传视频流会让成片只剩音频; ffmpeg-python 双 input 不能链式。

Step 5 | ex5_verify.py —— 成片体检 (finalize 环节)

操作: `python ex5_verify.py` 目的: 校验分镜表字段闭环 + 解码计数

实现: 逐帧检查 图/音/段/时长 四字段全非空; 用 null muxer 解码数帧

✅ 预期结果 (实测): 5 帧 × 4 字段全 PASS; 成片解码帧数: 846 (30fps × ~28.2s); `[ex5] PASS`。整条链路耗时不到 2 分钟, 0 元。



进阶方向: 把 ex2 的 PIL 占位图换成真·AI 配图 (配 DashScope 或 ComfyUI 的 key 跑上游原版流水线); 用 `uv run` 起上游 WebUI (`start_web.bat`) 在浏览器里跑同一条流水线; 或给上游提一条"自定义帧模板"的 PR。